

What the WASM

A short introduction / overview

By Felix Eisenmenger



Topics

- What is WASM?
- Short WASM History
- WASM today
- Beyond JavaScript
- Compilation & Loading Workflow
- A Practical example
- WASM Drawbacks
- Summary

What is WASM



What is WASM

WASM => WebAssembly

- low-level assembly-like language with compact binary format runs with near-native performance
- provides languages like C/C++, C#, Rust with a compilation target to run directly on the web without a Web Framework
- Sandboxed security
- designed to complement and run alongside JavaScript

Short WASM History



Short WASM History

- **2015: The Announcement**
 - Mozilla, Google, Microsoft, and Apple announcing a joint collaboration to create a new binary code format for the web.
- **2017: The MVP Launches**
 - MVP was officially completed
 - supported in all four major browsers (Firefox, Chrome, Safari, and Edge)
 - Developers could now compile C, C++, and Rust to run in the browser at near-native speed.
- **2019: The Great Escape (W3C & WASI)**
 - Official Standard: The W3C declared WebAssembly the fourth official language of the web (joining HTML, CSS, and JavaScript).
 - WebAssembly System Interface (WASI) was launched
 - This allows WASM to run outside the browser, directly on servers, edge networks, and IoT devices.
- **2023–Present: Maturity & The Component Model**
 - Native Garbage Collection (WasmGC): Added support for languages like Kotlin, Java, and Dart to run efficiently without shipping their own massive runtimes
 - Component Model: Allowed different languages to talk to each other seamlessly in a single WASM application

WASM today



WASM today

no longer just a "futuristic web technology."

- **Enables Heavy Web apps** like Figma, Adobe Photoshop Web, and AutoCAD to deliver desktop-class performance in a browser tab.
- **Enables Edge & Serverless Computing** like Cloudflare, Fastly, and AWS use WASM to spin up serverless functions in microseconds (compared to milliseconds for Docker containers).
- **Enables Plugins & Extensibility** Platforms like Envoy and Shopify use WASM to allow users to write safe, sandboxed plugins in any language.

Beyond JavaScript



Beyond JavaScript Complete UI Runtimes

Bypassing the DOM with QML

- **Traditional Way:**
 - Write the UI in HTML/CSS/JS, call WASM for heavy compute
- **The Qt / QML Way:**
 - Write the entire application – UI and business logic – in QML and C++/Python
- **Direct Rendering:**
 - The WASM binary bypasses the browser's document object model (DOM) entirely and draws pixels straight to a `<canvas>` via WebGL
- **True Cross-Platform:**
 - Write a native desktop app once, deploy it to Windows, Mac, Linux, iOS, Android, and the Web via the exact same codebase.

Beyond JavaScript

Complete UI Runtimes

A QML Example

```
// Backend.hpp
#pragma once
#include <QObject>
#include <QString>

class Backend : public QObject {
    Q_OBJECT

public:
    explicit Backend(QObject *parent = nullptr);

    // Q_INVOKABLE exposes this native C++ method directly to the QML engine
    Q_INVOKABLE QString computeHeavyHash(const QString &input);
};

/* ----- */

// Backend.cpp
#include "Backend.h"
#include <QCryptographicHash>

Backend::Backend(QObject *parent) : QObject(parent) {}

QString Backend::computeHeavyHash(const QString &input) {
    // Simulating a fast, native cryptographic operation inside the WASM sandbox
    QByteArray hash = QCryptographicHash::hash(input.toUtf8(),
    QCryptographicHash::Sha256);

    return QString(hash.toHex());
}
```

carbon
carbon.now.sh

the custom C++ file(s) and the declarative layout elements are **packed** safely together into **one** .wasm file

```
import QtQuick
import QtQuick.Controls

ApplicationWindow {
    width: 400; height: 350
    visible: true; color: "#121212"

    Column {
        anchors.centerIn: parent
        spacing: 25
        alignment: Qt.AlignHCenter

        // 1. The Button triggering the C++ logic
        Rectangle {
            id: customButton
            width: 220; height: 60; radius: 30
            color: buttonMouseArea.pressed ? "#deff9a" : "#1e1e1e"
            border.color: "#deff9a"; border.width: 2

            Behavior on color { ColorAnimation { duration: 150 } }

            Text {
                anchors.centerIn: parent
                text: "Run Native C++ Hash"
                font.pixelSize: 16; font.bold: true
                color: customButton.color == "#deff9a" ? "#000000" : "#ffffff"
            }

            MouseArea {
                id: buttonMouseArea
                anchors.fill: parent
                onClicked: {
                    // Call the C++ function directly and handle the return value in UI
                    let rawData = "SecureWasmInputData_2026";
                    let result = backend.computeHeavyHash(rawData);

                    hashDisplay.text = result;
                }
            }
        }

        // 2. Display element showing output data from the C++ side
        Text {
            id: hashDisplay
            text: "Click button to compute SHA-256"
            color: "#daffde"
            font.pixelSize: 14
            font.family: "Monospace"
            width: 340
            wrapMode: Text.WrapAnywhere
            horizontalAlignment: Text.AlignHCenter
        }
    }
}
```

carbon
carbon.now.sh

Compilation & Loading Workflow



Compilation & Loading Workflow

1. Source Code – some high-level code in a language you want (Rust, C++, Go, Python etc...)
2. Compilation – Compile the code with *wasm-pack* or *emscripten*
with a target defined like *wasm32-unknown-unknown* / *wasm32-wasi*
3. Output – a *.wasm* file and a *.js* "glue" file
4. Runtime – Browser fetches, compiles and instantiate the binary alongside JavaScript

A Practical example



A Practical example: PolyGlyph

Now we're looking at a staged example which only exists for showing the possibilities of using WASM.

Imagine a OCR Related Task but you're Team doesn't know what language should be used cause everyone has it's on preferences. And maybe you need to develop a Docker/Kubernetes infrastructure.

A Practical example: PolyGlyph

The Solution:

- | | |
|------------------------|--|
| 1. RUST Module: | Fast, memory-safe image preprocessing (grayscale/contrast) |
| 2. C++ Module: | Machine Learning Core (Local ONNX Runtime Inference) |
| 3. GO Module: | Enterprise Compliance (Regex PII data masking on tokens) |
| 4. PYTHON (Pyodide): | Data Science Block (Saves output to custom enterprise dataframe) |
| 5. React/Vue Frontend: | Easy shipping modern UI/UX |

A Practical example: PolyGlyph

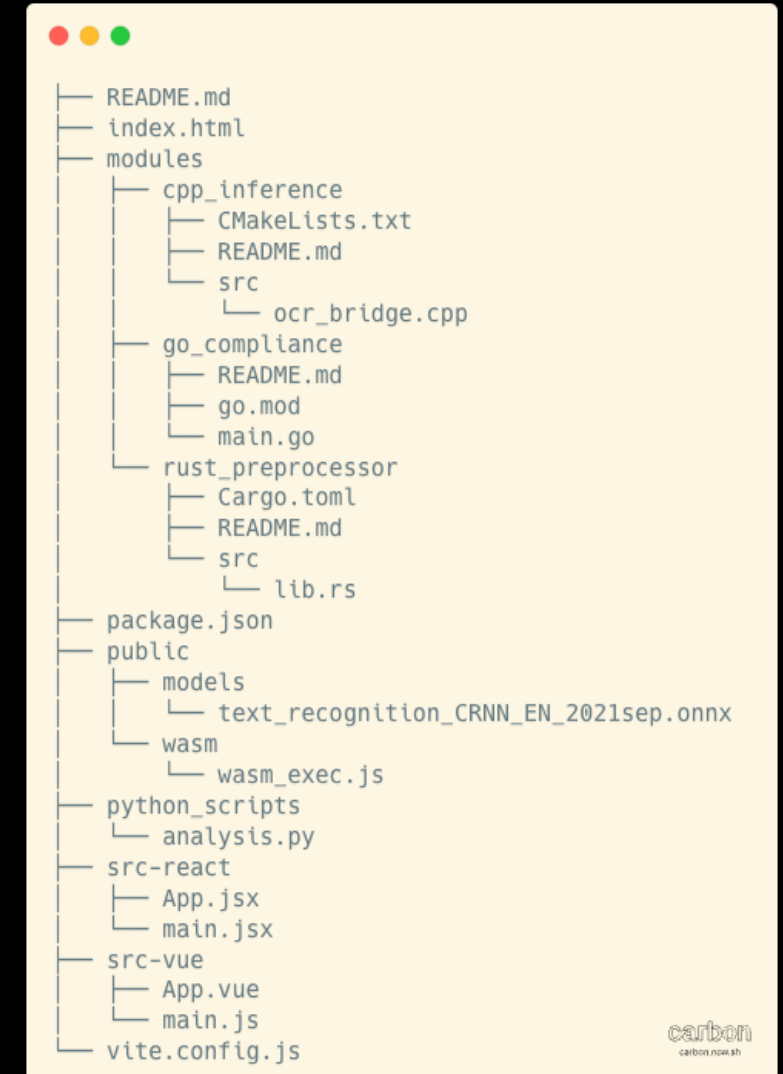
The Project structure:

- Avoids "language silos"
- No additional microservices, network latency or protobuf definitions

```
├── README.md
├── index.html
├── modules
│   ├── cpp_inference
│   │   ├── CMakeLists.txt
│   │   ├── README.md
│   │   └── src
│   │       └── ocr_bridge.cpp
│   ├── go_compliance
│   │   ├── README.md
│   │   ├── go.mod
│   │   └── main.go
│   └── rust_preprocessor
│       ├── Cargo.toml
│       ├── README.md
│       ├── src
│       │   └── lib.rs
│       └── wasm
│           └── wasm_exec.js
├── package.json
├── public
│   ├── models
│   │   └── text_recognition_CRNN_EN_2021sep.onnx
│   └── wasm
│       └── wasm_exec.js
├── python_scripts
│   └── analysis.py
├── src-react
│   ├── App.jsx
│   └── main.jsx
├── src-vue
│   ├── App.vue
│   └── main.js
└── vite.config.js
```

A Practical example: PolyGlyph

- Core engines are living under */modules*
 - Enables to split the tasks by language strength
- Modern Frontend */src-vue & /src-react*
 - For illustrating that a specific Framework is not required
- Data Brain */python_scripts & /public/models*
 - Separation of concerns and which data is loaded into the public folder



A Practical example: PolyGlyph

1. Rust: The Secure Image Preprocessor

- has native tooling (**wasm-bindgen**)
- reduces glue-code overhead to almost zero
- perfect for processing raw data safely without risks of buffer overflows

```
use wasm_bindgen::prelude::*;

#[wasm_bindgen]
pub fn preprocess_image_channels(pixels: &mut [u8], width: u32, height: u32) {
    // High-speed, memory-safe pixel manipulation
    for i in (0..pixels.len()).step_by(4) {
        let r = pixels[i] as f32;
        let g = pixels[i+1] as f32;
        let b = pixels[i+2] as f32;

        // Convert to grayscale to optimize for OCR text visibility
        let gray = (0.299 * r + 0.587 * g + 0.114 * b) as u8;
        pixels[i] = gray;
        pixels[i+1] = gray;
        pixels[i+2] = gray;
    }
}
```

A Practical example: PolyGlyph

2. C++: Machine Learning Core

- performance and legacy code reuse
- highly optimized native codebases
- executes compiled, bare-metal assembly instructions via the WASM virtual machine
- can load and manipulate large binary models (~16MB–100MB+) directly in WebAssembly

```
#include <iostream>
#include <vector>
#include <string>
#include <algorithm>
#include <onnxruntime_cxx_api.h>
#include <emscripten.h>

// The 36-character vocabulary used by OpenCV's English CRNN model
const std::string CHARSET = "0123456789abcdefghijklmnopqrstuvwxyz";

// Global string buffer to safely hold text for JavaScript extraction
std::string g_output_text;

// Basic CTC Greedy Decoder: takes raw model probabilities and turns them into text
std::string ctc_decode(const float* output_data, int sequence_length, int alphabet_size) {
    std::string text = "";
    int prev_class_idx = -1;

    for (int i = 0; i < sequence_length; ++i) {
        int max_class_idx = 0;
        float max_prob = output_data[i * alphabet_size];

        // Find character with highest probability for this time-step (ArgMax)
        for (int j = 1; j < alphabet_size; ++j) {
            float prob = output_data[i * alphabet_size + j];
            if (prob > max_prob) {
                max_prob = prob;
                max_class_idx = j;
            }
        }

        // In CTC, the last index (alphabet_size - 1) is reserved for the "blank" token
        if (max_class_idx != (alphabet_size - 1) && max_class_idx != prev_class_idx) {
            text += CHARSET[max_class_idx];
        }
        prev_class_idx = max_class_idx;
    }
    return text;
}
```

```
extern "C" {

    EMSCRIPTEN_KEEPALIVE
    uint8_t* allocate_wasm_buffer(int size) { return new uint8_t[size]; }

    EMSCRIPTEN_KEEPALIVE
    void free_wasm_buffer(uint8_t* ptr) { delete[] ptr; }

    EMSCRIPTEN_KEEPALIVE
    const char* process_image_to_string(uint8_t* rgba_data, int width, int height) {

        // 1. Preprocess: OpenCV CRNN expects a single-channel Grayscale float input
        // Model dimensions required by OpenCV Zoo: 100x32 (Width x Height)
        int target_width = 100;
        int target_height = 32;
        std::vector<float> input_tensor_values(target_width * target_height, 0.0f);

        // Simple bilinear resize & grayscale conversion mapping into the 100x32 buffer
        for (int y = 0; y < target_height; ++y) {
            for (int x = 0; x < target_width; ++x) {
                int src_x = x * width / target_width;
                int src_y = y * height / target_height;
                int src_idx = (src_y * width + src_x) * 4;

                // Convert RGBA to Grayscale value normalized between 0.0 and 1.0
                float r = rgba_data[src_idx + 0];
                float g = rgba_data[src_idx + 1];
                float b = rgba_data[src_idx + 2];
                float gray = (0.299f * r + 0.587f * g + 0.114f * b) / 255.0f;

                input_tensor_values[y * target_width + x] = gray;
            }
        }

        // 2. Inference setup
        static Ort::Env env(Ort::LoggingLevel::WARNING, "OpenCWOcr");
        static Ort::SessionOptions session_options;
        static Ort::Session session(env, "models/text_recognition_CRNN_EN_2021sep.onnx", session_options);

        // Model shape input format: [Batch, Channels, Height, Width] -> [1, 1, 32, 100]
        std::vector<int64_t> input_shape = {1, 1, target_height, target_width};
        auto memory_info = Ort::MemoryInfo::CreateCpu(OrtArenaAllocator, OrtMemTypeDefault);

        Ort::Value input_tensor = Ort::Value::CreateTensor<float>(
            memory_info, input_tensor_values.data(), input_tensor_values.size(),
            input_shape.data(), input_shape.size()
        );

        // Input and Output layer names from the OpenCV ONNX file configuration
        const char* input_names[] = {"input"};
        const char* output_names[] = {"output"};

        auto output_tensors = session.Run(
            Ort::RunOptions{nullptr},
            input_names, &input_tensor, 1,
            output_names, 1
        );

        // 3. Extract output shape parameters
        // Output tensor shape is [Sequence Length (typically 25), Batch (1), Vocabulary Size (37)]
        auto output_shape = output_tensors[0].GetTensorTypeAndShapeInfo().GetShape();
        int seq_len = output_shape[0];
        int alphabet_size = output_shape[2];

        float* raw_output = output_tensors[0].GetTensorMutableData<float>();

        // 4. Decode the text natively using C++ CTC algorithm
        g_output_text = ctc_decode(raw_output, seq_len, alphabet_size);

        // Return C-string pointer directly across the WASM boundary to JavaScript
        return g_output_text.c_str();
    }
}
```

A Practical example: PolyGlyph

3. Go: The Enterprise Compliance Guard

- TinyGo can compile existing validation libraries, routing rules, and compliance algorithms directly to the frontend.

```

package main

import (
    "regexp"
    "syscall/js"
)

// Exported function to JavaScript
func maskSensitiveData(this js.Value, args []js.Value) interface{} {
    text := args[0].String()

    // Enterprise Regex to find and mask credit card numbers locally
    re := regexp.MustCompile(`\b\d{4}[- ]?\d{4}[- ]?\d{4}[- ]?\d{4}\b`)
    maskedText := re.ReplaceAllString(text, "[REDACTED_CARD]")

    return maskedText
}

func main() {
    // Block execution and register function in JS global scope
    c := make(chan struct{}, 0)
    js.Global().Set("maskSensitiveData", js.FuncOf(maskSensitiveData))
    <-c
}

```

A Practical example: PolyGlyph

4. Python (Pyodide): Client-Side Data Science

- full Python runtime compiled to WASM
- No rewrite for Pandas or NumPy scripts
- Easy usage

```
// Load the Pyodide WASM Runtime
let pyodide = await loadPyodide();

// Run a raw Python data analysis script over the text inside the browser
let pythonScript = `
def analyze_text(ocr_output):
    words = ocr_output.split()
    metrics = {
        "word_count": len(words),
        "is_invoice": "invoice" in ocr_output.lower()
    }
    return metrics
`;
await pyodide.runPythonAsync(pythonScript);
const analyze = pyodide.globals.get('analyze_text');
const results = analyze(finalOcrText);
```

A Practical example: PolyGlyph

5. Shared Vite Configuration + package.json:

- Unified compilation steps
- Easy to integrate into other Frameworks

```

{
  ...
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview",
    "--- WASM COMPILATION SCRIPTS ---": "",
    "build:rust": "cd modules/rust_preprocessor && wasm-pack build --target web --out-dir ../../public/wasm/rust",
    "build:go": "cd modules/go_compliance && tinygo build -o ../../public/wasm/go_compliance.wasm -target wasm main.go && cp $(tinygo
env TINYGO_ROOT)/targets/wasm_exec.js ../../public/wasm/",
    "build:cpp": "cd modules/cpp_inference && mkdir -p build && cd build && emcmake cmake .. && emmake make",
    "build:all-wasm": "npm run build:rust && npm run build:go && npm run build:cpp"
  },
  "dependencies": {
    ...
  },
  "devDependencies": {
    "vite-plugin-top-level-await": "^1.4.0",
    "vite-plugin-wasm": "^3.3.0"
  }
}

```

```

import { defineConfig } from 'vite';
import react from '@vitejs/react-refresh';
import vue from '@vitejs/plugin-vue';
import wasm from 'vite-plugin-wasm';
import topLevelAwait from 'vite-plugin-top-level-await';

export default defineConfig({
  plugins: [
    react(),
    vue(),
    wasm(),
    topLevelAwait()
  ],
  server: {
    headers: {
      // Required security profiles to unlock local high-performance WebAssembly memory buffer threads
      'Cross-Origin-Opener-Policy': 'same-origin',
      'Cross-Origin-Embedder-Policy': 'require-corp',
    },
  },
});

```

A Practical example: PolyGlyph

6. Simplified Frontend Integration (React & Vue):

- Reducing the need for additional heavy frontend npm packages
- all run within the same web worker/browser tab thread
- Data transfers between different modules take a fraction of milliseconds
- can directly be used within multiple Frontend Frameworks

```
import initRust, { preprocess_image_channels } from './wasm/rust/rust_preprocessor.js';

async function handleImageProcess(file) {
  // 0. Ensure Rust is initialized (usually done once on component mount)
  await initRust();

  // 1. Extract raw RGBA pixels from the uploaded image using an HTML Canvas
  const img = await loadImageElement(file);
  const canvas = document.createElement('canvas');
  const ctx = canvas.getContext('2d');
  canvas.width = img.width;
  canvas.height = img.height;
  ctx.drawImage(img, 0, 0);

  const imageData = ctx.getImageData(0, 0, canvas.width, canvas.height);
  const rawPixels = imageData.data; // Uint8ClampedArray (RGBA)

  // =====
  // STEP 1: RUST PREPROCESSING
  // =====
  // Rust modifies the rawPixels array in-place via high-speed WebAssembly memory.
  // It normalizes the image and isolates text channels to maximize OCR accuracy.
  preprocess_image_channels(rawPixels, canvas.width, canvas.height);

  // =====
  // STEP 2: C++ MACHINE LEARNING CORE (OpenCV CRNN ONNX)
  // =====
  // Allocate a buffer inside the C++ WASM linear memory heap
  const wasmBufferPtr = Module._allocate_wasm_buffer(rawPixels.length);

  // Copy the RUST-OPTIMIZED pixels directly into the C++ WASM heap
  Module.HEAPU8.set(rawPixels, wasmBufferPtr);

  // Execute C++ inference + native CTC decoding
  // This returns a pointer to the final decoded string sitting in C++ memory
  const cStringPtr = Module._process_image_to_string(wasmBufferPtr, canvas.width, canvas.height);

  // Read the string directly out of the C++ WASM Heap into JavaScript
  const rawTextFromOcr = Module.UTF8ToString(cStringPtr);

  // Free the C++ heap buffer to prevent memory leaks
  Module._free_wasm_buffer(wasmBufferPtr);

  // =====
  // STEP 3: GO ENTERPRISE COMPLIANCE
  // =====
  // Pass the text string into the TinyGo module to scrub PII / Credit Cards
  const safeText = window.maskSensitiveData(rawTextFromOcr);

  // =====
  // STEP 4: PYTHON DATA SCIENCE METRICS
  // =====
  // Pass the safe text to Pyodide for structural analysis (if using the full layout)
  // const metrics = window.analyzeTextWithPython(safeText);

  return safeText;
}
```


A Practical example: PolyGlyph

Benefits of this architecture.

- Cost Savings:
 - No spinning up of clusters with expensive GPUs
 - Workload is completely offloaded to the user's local machine for free
- Ultimate Security & Privacy:
 - Data compliance engine (Go), ML engine (C++), image processor (Rust)
run full client-side in a secure WASM sandbox, sensitive data (images) never leave the user's machine
 - Achieving easier high HIPAA, GDPR compliance
- Unified Codebase:
 - Every Team can stick to their preferred language
 - Languages can be picked on best usage for a certain task

A Practical example: PolyGlyph

Drawbacks of this architecture.

- Toolchain Tax
 - Not all C++/CMake libraries are portable to WASM cause of POSIX threads or OS file system access (the WASM sandbox has only a virtual file system)
 - Some Rust crates uses legacy C libraries under the hood which are not (easily) compileable to WASM
 - Go is designed for servers not compact binaries which can lead to Binary bloating when not setup carefully
 - Python needs its own CPython interpreter when no other libraries like Pyodide are used
- Debug Hell
 - Even when every individual module is tested carefully when it comes to problems during production Error detection can be an extensive task and requires high team communication / understanding of each language
 - Error Messages inside of the browser are unreadable, obfuscated WASM memory address

WASM Drawbacks



WASM Drawbacks

32-bit Architecture

- For Standard WASM
- Limits linear memory to 4GB
(can be a hurdle for data-heavy C++ apps)
-

JS Interop Overhead

- Passing objects between requires serialization and data copy
- Can negate the performance when used too frequent
-

Binary Bloat

- Languages which ship their own standard library or runtimes lead to bigger .wasm files
- Can slow down initial page loading compared to lightweight JS
-

Summary



Summary

"If WASM+WASI existed in 2008, we wouldn't have needed to create Docker. That's how important it is."

– Solomon Hykes, Co-founder of Docker

- WASM didn't come to kill JavaScript; it came to give it a superpower
- It started as a tool to make web browsers faster, but it has evolved into a secure, language-agnostic, ultra-fast sandbox for cloud and edge computing
- When creating additional microservices consider using WASM as optional choice

Thanks

for listening



Time for your questions